

Clojure マクロを読み、書き、控える

評価・衛生・DSL までを一冊で身につける教科書

LearnMacro

目次

Clojure マクロを読み、書き、控える	3
奥付	4
ライセンス	4
環境	4
制作物の場所	4
正誤・更新	4
免責	4
通し原稿（完全版）	4
はじめに（確定版 v1.0）	5
対象読者	5
通読版の読み方	5
完全版の読み方	5
本書の約束	5
第 1 章 イントロダクション—関数では届かないこと	5
1.1 動機	6
1.2 対照実験: and を関数で書く	6
1.3 マクロが解く問題の型	6
1.4 演習（判断）	6
1.5 まとめ—関数でよいのか？	7
第 2 章 コードはデータ—読み込みと評価	7
2.1 動機	7
2.2 段階	7
2.3 フォームを手で組む	7
2.4 特殊形式・マクロ・関数	7
2.5 演習	8
第 3 章 クォート、構文引用、アンクォート	8
3.1 動機	8
3.2 quote / '	8
3.3 構文引用 (syntax-quote) `	8
3.4 アンクォート ~ とスプライシング ~@	8
3.5 自動 gensym foo#（予告）	9
3.6 よくある混乱	9
3.7 演習	9
第 4 章 はじめての defmacro	9
4.1 動機	9

4.2 最小のマクロ: <code>unless</code>	9
4.3 関数ではなぜ足りないか	10
4.4 ドキュメント	10
4.5 演習	10
4.6 まとめ—関数でよいのか?	10
第5章 展開を見る・読む・試す	10
5.1 動機	10
5.2 道具	11
5.3 予想 → 展開 → 差分	11
5.4 <code>&form / &env</code> (通読版は存在だけ)	11
5.5 罫: 展開確認用フォームの作り方	11
5.6 演習	11
第6章 衛生、gensym、二重評価	12
6.1 動機	12
6.2 二重評価	12
6.3 変数捕獲	13
6.4 演習 (直す)	13
6.5 まとめ—関数でよいのか?	13
第7章 標準マクロを分解する (抜粋)	13
7.1 動機	13
7.2 <code>when</code>	13
7.3 <code>or</code> (短絡)	13
7.4 <code>when-let</code>	13
7.5 通読版での止め方と追記	14
7.6 演習	14
第8章 制御構文を自分で書く—<code>unless</code> 再訪	14
8.1 動機	14
8.2 設計手順 (再掲)	14
8.3 再確認	14
8.4 やりすぎの兆候	15
8.5 演習	15
第9章 束縛とリソースのマクロ	15
9.1 動機	15
9.2 束縛マクロの定石	15
9.3 <code>if-let-lite / when-let-lite</code>	16
9.4 リソース管理— <code>with-open</code> の構造	16
9.5 <code>with-redefs</code> の位置づけ	16
9.6 遅延の糖衣 (橋渡し)	16
9.7 定義マクロへの橋渡し	17
9.8 演習	17
9.9 まとめ—関数でよいのか?	17
第10章 コード生成の定石とヘルパー関数への委譲	17
10.1 動機	17
10.2 テンプレート展開 (素直な形)	17
10.3 薄いマクロ+関数	17
10.4 データからコードへ (テーブル駆動)	18
10.5 フォームの行走査— <code>clojure.walk</code>	18
10.6 マクロがマクロを生成する	18
10.7 コンパイル時計算	18
10.8 演習	18

10.9 まとめ—関数でよいか？	19
第 11 章 DSL の設計判断	19
11.1 動機	19
11.2 内部 DSL の種類	19
11.3 判断基準	19
11.4 良い境界—入口だけマクロ、中はデータ+関数	19
11.5 事例観察（対比）	20
11.6 悪い DSL の兆候	20
11.7 小さな DSL スケッチ	20
11.8 演習	20
11.9 まとめ—関数でよいか？	21
第 12 章 テスト、エラー、ツールチェーン	21
12.1 動機	21
12.2 何をテストするか	21
12.3 展開の単位テスト	21
12.4 エラーメッセージ	21
12.5 リンタ・IDE（概要）	22
12.6 バージョンと再現	22
12.7（補）ClojureScript	22
12.8 演習	22
12.9 まとめ—関数でよいか？	22
第 13 章 ケーススタディ（A/B/C）	22
13.1 進め方	22
13.2 ケース A—unless / when-let-lite	23
13.3 ケース B—labeled-time	23
13.4 ケース C—cond→lite（完全版）	24
13.5 振り返り	25
13.6 演習	25
第 14 章 アンチパターンとチェックリスト（通読版短縮）	26
14.1 動機	26
14.2 書く前	26
14.3 書いた後	26
14.4 よくある失敗（短く）	26
14.5 演習（判断）	26
付録 A—ClojureScript 補注	27
A.1 まず押さえること	27
A.2 JVM Clojure との差（実務で刺さる点）	27
A.3 やってよい／控える（CLJS 文脈）	28
A.4 最小スケッチ（イメージ）	28
A.5 この付録の止め方	28

Clojure マクロを読み、書き、控える

副題: 評価・衛生・DSL までを一冊で身につける教科書

読める・書ける・控えられる——展開を見て保守し、関数とデータの境界を先に決める教科書。

- **版:** 確定版 v1.0（工程 11）
- **対象環境:** Clojure 1.12.x / 推奨 JDK 21
- **読み順（完全版）:** はじめに → 第 1-14 章
- **読み順（通読版）:** はじめに → 1-8 → 13(A/B) → 14

- **リポジトリ:** <https://github.com/ichisemasashi/LearnMacro>

奥付

書名: Clojure マクロを読み、書き、控える
副題: 評価・衛生・DSL までを一冊で身につける教科書
版: 確定版 v1.0 (工程 11)
発行日: 2026-09-11
リポジトリ: [ichisemasashi/LearnMacro](https://github.com/ichisemasashi/LearnMacro)

ライセンス

対象	ライセンス
サンプルコード (src//test//exercises/answers)	../LICENSE
本文・原稿・企画・ノート	../LICENSE-TEXT.md

環境

- Clojure **1.12.1** (deps.edn)
- 推奨 JDK **21** (検証実測は環境により異なる場合あり)

制作物の場所

- HTML / EPUB / PDF: dist/
- 通し原稿: 原稿/通し原稿 _ 完全版.md / 原稿/通し原稿 _ 通読版.md
- ビルド手順: 制作/build.sh (PDF は Noto Sans CJK JP + JetBrains Mono)
- 商業分岐: ./商業入稿ガイド.md / ./表紙 _ 商業案.md

正誤・更新

- 正誤表: ../正誤表.md
- 変更履歴: ../CHANGELOG.md
- フィードバック: GitHub Issues (上記リポジトリ)

免責

本書およびサンプルは学習目的で提供する。本番利用前に各自で検証すること。

通し原稿 (完全版)

版: 確定版 v1.0 + 次版追記 (N-09/N-10) **題名:** Clojure マクロを読み、書き、控える **副題:** 評価・衛生・DSL までを一冊で身につける教科書

章ファイルを通読順に連結。編集の正本は各章ファイル。目次: **目次.md** / 用語集: **用語集.md** / 索引: **索引.md** / 図表: **図表一覧.md** / 付録: **付録_CLJS 補注.md**

はじめに（確定版 v1.0）

題名: Clojure マクロを読み、書き、控える

副題: 評価・衛生・DSL までを一冊で身につける教科書

本書は、Clojure のマクロを **読める・書ける・控えられる** ようになるための通読書である。

「すごい構文」を集める本ではない。展開結果を見て保守し、関数やデータで足りるときはマクロを選ばない判断まで含める。

対象読者

Clojure の基本（関数・コレクション・名前空間・if/let/fn/def）を一通り使える人。

通読版の読み方

初版の「薄い一本」は次の順である。

1 → 2 → 3 → 4 → 5 → 6 → 7（抜粋）→ 8 → **13**（ケース A/B）→ 14

第 9-12 章は通読版では飛ばす（束縛の深掘り、コード生成、DSL、ツールは完全版）。番号が飛ぶのはそのためである。

完全版の読み方

通読版に続けて（または第 8 章の次から）次を読む。

1 → … → 8 → **9 → 10 → 11 → 12** → 13 → 14

目次: **目次.md**

サンプルコードはリポジトリで動く。

```
clojure -M:run
clojure -M:test
```

対象環境: Clojure **1.12.x**（サンプルは deps.edn の **1.12.1**）。推奨 JDK は **21**（検証実測は環境により異なる場合あり）。

展開例に出てくる gensym 名（temp__123__auto__ 等）は実行ごとに変わるので、形だけを追う。

用語は **用語統一表.md** に従う。定義は **用語集.md**、章一覧は **目次.md**。

通し読み: **通し原稿 _ 通読版.md**（薄い一本）／**通し原稿 _ 完全版.md**（1-14 通し）。ClojureScript との差分だけは **付録 _ CLJS 補注.md** を参照（本文は JVM Clojure 前提）。

本書の約束

1. clojure.core と、対比用の定番ライブラリ入口を、展開や設計境界付きで読める
2. 制御構文・束縛・薄い糖衣を、衛生に配慮して書ける
3. 「関数／データで足りる」と判断できる

詳細な企画は 企画提案書.md、目次の正本は 構成設計.md を参照してほしい。

第 1 章 イントロダクション—関数では届かないこと

状態: 確定版 v1.0（工程 11） **サンプル:** learn-macro.ch01-intro

1.1 動機

Clojure では、ほとんどの抽象は関数で足りる。それでもマクロが残る理由は単純で、**引数の評価を呼び出し側の都合で止めたり、並べ替えたりできる**からである。関数は呼ばれた時点で引数がすでに値になっている。

1.2 対照実験: and を関数で書く

次の「関数版 and」は、見かけは似ていても短絡しない。

```
;; 関数は両引数を先に評価する
((fn [a b] (if a b a))
 (do (println "left") false)
 (do (println "right") :ran))
;; ⇒ left と right の両方が印字される
```

本物の and は左が偽なら右を評価しない。

```
(and (do (println "left") false)
      (do (println "right") :ran))
;; ⇒ left だけ
```

展開の一端は次のとおりである（環境により gensym 名は異なる）。

```
;; before
(and false :x)

;; after macroexpand-1 (概念)
(let [and__auto__ false]
  (if and__auto__ (and :x) and__auto__))
```

リポジトリでは clojure -M:run の Ch01 で副作用の差を確認できる。

1.3 マクロが解く問題の型

1. **評価の制御** — 短絡、遅延、条件付き実行
2. **構文の拡張** — 新しい制御構造・束縛
3. **コード形状の変換** — 見た目を領域向けにし、中身は通常の Clojure へ
4. (慎重に) **コンパイル時の定型生成**

対して、ただの計算やコレクション変換は関数の仕事である。

1.4 演習（判断）

次をマクロ／関数のどちらかに分類し、一文で理由を書け。

1. 値を印字して返すデバッグ用 API
2. lookup が non-nil のときだけ本体を走らせる束縛
3. マップのキーをキーワード化する純変換

解答例: src/learn_macro/answers/ch01.clj

1.5 まとめ—関数でよいか？

短絡や「まだ評価したくない式」が本質ならマクロ（または特殊形式）の領域に近い。
値がすでに手元にあり、変換するだけなら関数でよい。

マクロを増やしすぎると呼び出し側にも特殊な書き方を強いることがある（**マクロ感染**）。判断のチェックリストは [第 14 章](#) でまとめる。

第 2 章 コードはデータ—読み込みと評価

状態: 確定版 v1.0（工程 11） サンプル: learn-macro.ch02-data

2.1 動機

マクロを理解する鍵は、「プログラムのテキスト」と「評価された値」のあいだに、**フォームというデータ**があることだ。マクロはそのデータを受け取り、別のフォームを返す。

2.2 段階

おおまかには次の流れである。

1. **Read** —テキスト → データ（リスト・シンボル・リテラル等）
2. **Macroexpand** —マクロ呼び出しを、別のフォームへ書き換え
3. **Compile / Eval** —実行可能な形へ／値を得る

「いつ形が変わるか」を意識すると、macroexpand の結果と実行結果を混同しにくくなる。

2.3 フォームを手で組む

```
(def form (list '+ 1 2))  
(first form)    ; ⇒ +  
(rest form)     ; ⇒ (1 2)
```

これは「コードをデータとして触っている」状態である。
eval すれば 3 になるが、eval は**マクロの代わりではない**。タイミングも、必要な文脈も、安全モデルも違う。
本書ではデモ以外で eval に頼らない。

2.4 特殊形式・マクロ・関数

表 2.1 呼び出しの種類と評価

種類	引数の評価	拡張
特殊形式	独自規則	言語組み込み
マクロ	まず展開（引数は未評価のフォーム）	ユーザが追加可
関数	すべて評価してから本体	ユーザが追加可

エラーや奇妙な挙動に出会ったら、まずこのどれかを切り分ける。

2.5 演習

1. **読む**: (when true (println 1) 2) が、read / expand / eval のどこで何になるかラベルを付けよ。
2. **書く**: list だけで (+ 1 2) 相当のフォームを組み立て、eval せずに構造だけ検査せよ。

たたき台: src/learn_macro/exercises/ch02.clj / 解答: src/learn_macro/answers/ch02.clj ##
2.6 まとめ—関数でよいか？

データ変換としてコードに触る必要がなければ、無理にフォームを組むな。
マクロは「評価前の形」を扱う道具であり、通常のビジネスロジックの置き場ではない。

実行: (learn-macro.ch02-data/demo)

第3章 クォート、構文引用、アンクォート

状態: 確定版 v1.0 (工程 11) **サンプル**: learn-macro.ch03-quoting

3.1 動機

マクロの本体は、多くの場合 **テンプレート** でフォームを組み立てる。そのテンプレート言語がクォート族である。

3.2 quote / '

```
'(+ 1 2)
;; ⇒ (+ 1 2) ; 評価されないリスト
```

3.3 構文引用 (syntax-quote) `

```
`(+ 1 2)
;; ⇒ (clojure.core/+ 1 2) ; 名前が名前空間修飾される
```

修飾が付くことで、展開先でも意図した + を指しやすくなる。

3.4 アンクォート ~ とスプライシング ~@

```
(let [x 10
      ys [1 2 3]]
  `(+ ~x 2)      ; ⇒ (clojure.core/+ 10 2)
  `(+ ~@ys))     ; ⇒ (clojure.core/+ 1 2 3)
```

- ~ …テンプレート内に、評価結果を 1 個埋め込む (**アンクォート**)
- ~@ …列を親リストに繋ぎ込む (**スプライシング**)

表 3.1 クォート族の役割

記号	呼び名	役割
' / quote	クォート	評価しないデータとして読む
`	構文引用	修飾付きテンプレート
~	アンクォート	テンプレートの穴 (1 値)
~@	スプライシング	列を親へ繋ぎ込む

標準用語は表 3.1 とおり。混同しやすいので「反引用」は使わない（用語統一表.md）。

3.5 自動 gensym foo# (予告)

```
`(let [a# 1] a#)
;; ⇒ (clojure.core/let [a__123__auto__ 1] a__123__auto__)
```

同一の構文引用内では同じ a# が同じシンボルになる。

安全に使う演習は第 6 章まで待つ。ここでは「一意な名前が付く」ことだけ覚える。

3.6 よくある混乱

表 3.2 よくある混同

混同	実際
' と ` は同じ ~ の外でも評価が止まる	修飾と ~ の可否が違う ~ の中だけが「穴」

3.7 演習

1. **書く:** 局所 x とベクタ ys から `(str ~x ~@ys)` 相当を作れ。
2. **直す:** ~ と ~@ を取り違えたテンプレートを直せ。

たたき台: src/learn_macro/exercises/ch03.clj / 解答: src/learn_macro/answers/ch03.clj ##
3.8 まとめ—関数でよいのか？

すでに値が手元にあり、単に呼び出すだけならクォートは不要。

まだ評価したくない形をデータとして保持・変形するとき、クォートが必要になる。

実行: (learn-macro.ch03-quoting/demo)

第 4 章 はじめての defmacro

状態: 確定版 v1.0 (工程 11) **サンプル:** learn-macro.ch04-defmacro

4.1 動機

クォートでフォームを組めるようになったら、次はそれを **呼び出しのたびに差し込む** 仕組みが defmacro である。

4.2 最小のマクロ: unless

```
(defmacro unless
  "test が偽のとき body を評価する。"
  [test & body]
  `(if (not ~test)
      (do ~@body)))
```

ポイント:

- 引数 test や body は **まだ評価されていないフォーム**
- 戻り値は、コンパイラが代わりに使う **展開後フォーム**

展開前後

```
;; before
(unless false :ok)

;; after macroexpand-1
(if (clojure.core/not false) (do :ok))
```

実行

```
(unless true :no) ; => nil (本体は走らない)
(unless false :yes) ; => :yes
```

4.3 関数ではなぜ足りないか

```
(defn unless-fn [test body-fn]
  (when-not test (body-fn)))
```

```
(unless-fn false (fn [] :yes))
```

動くが、呼び出し側が常に (fn [] ...) を強いられる。

構文として unless と書きたいならマクロ（または同等の特殊形式）が要る。

4.4 ドキュメント

defmacro にも docstring を付ける。利用者と未来の自分が展開の意図を追うときの手がかりになる。

4.5 演習

1. **書く:** when-not-lite (when-not 相当) を実装し、展開と実行を確認せよ。
2. **判断:** その糖衣は、チームのコードベースで関数 API のままの方がよいか？

骨組み: src/learn_macro/exercises/ch04.clj / 解答: src/learn_macro/answers/ch04.clj

4.6 まとめ—関数でよいのか？

呼び出し側に遅延 (thunk) を強いてもよいなら関数で足りることが多い。
言語に新しい分岐構文を足したいときだけ defmacro を検討する。

実行: (learn-macro.ch04-defmacro/demo)

第 5 章 展開を見る・読む・試す

状態: 確定版 v1.0 (工程 11) **サンプル:** learn-macro.ch05-expand

5.1 動機

マクロのデバッグで最初にやることは、実行結果をいじることより **展開結果を見ること** である。

5.2 道具

表 5.1 展開を見る関数

関数	意味
macroexpand-1 macroexpand	先頭がマクロなら 一段だけ 展開 先頭がマクロのあいだ繰り返し (入れ子の内側は見ない)
clojure.walk/macroexpand-all	木を歩き、内側も展開 (限界あり)

```
(macroexpand-1 '(when true 1 2))  
;; => (if true (do 1 2))  
  
(clojure.walk/macroexpand-all  
  '(when true (when false 1)))  
;; => (if true (do (if false (do 1))))
```

5.3 予想 → 展開 → 差分

ワークシートの型:

1. 手で展開形を書く
2. macroexpand-1 する
3. 差分を一言でメモする

この型を、後述の第 7 章「標準マクロを読む」でも使う。

コラム 5.A — 展開結果の読み方

gensym 名 (temp__123__auto__ など) は実行ごとに変わる。比較するときは「束縛が一度あるか」「再帰が残っているか」など **形** を見る。= で厳密一致させるテストでは、自分で書いたマクロ側の x# を固定するか、先頭シンボルと骨格だけを検査するとよい。

5.4 &form / &env (通読版は存在だけ)

defmacro 内では特殊な名前 &form (呼び出し全体) と &env (局所束縛の情報) が使える。通読版では「ある」と知るだけでよい。macroexpand だけで &env を覗くと空に見えることがあり、実コンパイル時と差が出る (../notes/核検証メモ.md)。

5.5 罫: 展開確認用フォームの作り方

`(my-macro [x ...]) のように **構文引用で引数まで書く** と、x が先に名前空間修飾され、展開結果が本番とずれる。

確認は次のようにする。

```
(macroexpand-1 (list `my-macro '[x (first [1])] '(inc x)))
```

5.6 演習

1. **読む**: (or false nil :ok) の展開を予想してから照合せよ。
2. **書く**: 自作 unless の展開テストを 1 本書け (= でフォーム比較でよい)。

たたき台: `src/learn_macro/exercises/ch05.clj` / 解答: `src/learn_macro/answers/ch05.clj` ##
5.7 まとめ—関数でよいか？

展開を見る習慣はマクロ専用の技能である。
関数のデバッグに `macroexpand` は不要——そのときは値とスタックを見よ。

実行: `(learn-macro.ch05-expand/demo)`

第 6 章 衛生、gensym、二重評価

状態: 確定版 v1.0 (工程 11) サンプル: `learn-macro.ch06-hygiene`

6.1 動機

マクロはフォームを繋ぎ合わせる。そのとき名前が衝突したり、同じ式が二度評価されたりする。
Clojure のマクロは **デフォルトでは衛生的ではない** (Common Lisp と同様。Scheme の `hygienic macro` とは違う)。
名前空間修飾と `gensym` / `foo#` で **自分で守る**。

6.2 二重評価

悪い発想 (教材用・骨格の完全形。完成 API としては載せない) :

```
(defmacro bad-time-double
  "【載せない失敗例】expr が二度現れる。"
  [expr]
  `(let [start# (System/nanoTime)]
    {:ms (/ (double (- (System/nanoTime) start#)) 1.0e6)
      :ret ~expr
      :ret-again ~expr}))
```

`expr` に副作用があると二度走る。
定石は **一度 `let` に束縛してから使う** ことである。

```
(defmacro good-time [expr]
  `(let [start# (System/nanoTime)
        ret# ~expr]
    {:ms (/ (double (- (System/nanoTime) start#)) 1.0e6)
      :ret ret#}))
```

`start#` / `ret#` は `auto-gensym` で、利用者の局所名とぶつかりにくい。

展開の骨格

```
;; before
(good-time (+ 1 2))

;; after (概念)
(let [start__auto__ (System/nanoTime)
      ret__auto__ (+ 1 2)]
  {:ms ... :ret ret__auto__})
```

6.3 変数捕獲

展開結果に、利用者が束縛している名前や、マクロ作者がベタ書きした名前が意図せず混ざるとバグになる。対策の基本は gensym または foo# である。

意図的に it などキャプチャする anaphoric マクロは、本書では推奨しない（賛否付きの深掘りは選択ケース／付録候補）。

6.4 演習（直す）

broken-twice は式を二度埋め込む。once-add として一度だけ評価する版を書け。

- たたき台: src/learn_macro/exercises/ch06.clj
- 解答: src/learn_macro/answers/ch06.clj

6.5 まとめ—関数でよいか？

二重評価も捕獲も、マクロ特有のコストである。
同じことを高階関数で書けるなら、その方がデバッグもテストも楽なことが多い。

実行: (learn-macro.ch06-hygiene/demo)

載せない例の一覧: ../notes/載せない失敗例.md

第 7 章 標準マクロを分解する（抜粋）

状態: 確定版 v1.0（工程 11） サンプル: learn-macro.ch07-core-read

7.1 動機

自分で書く前に、clojure.core の短いマクロから **型を盗む**。
読み方は第 5 章と同じく「予想 → macroexpand-1 → メモ」である。

7.2 when

```
(macroexpand-1 '(when true :a :b))  
;; => (if true (do :a :b))
```

盗む点: 複数ボディを do にまとめる薄い糖衣。

7.3 or（短絡）

```
(macroexpand-1 '(or false nil :ok))  
;; => (let [or__auto__ false]  
;;      (if or__auto__ or__auto__ (or nil :ok)))
```

盗む点: 一度束縛してから分岐し、**残りの or を再帰的に展開**する。二重評価を避けつつ短絡する。

7.4 when-let

```
(macroexpand-1 '(when-let [x (first [1])] (inc x)))  
;; => (let [temp__auto__ (first [1])]  
;;      (when temp__auto__  
;;        (let [x temp__auto__] (inc x))))
```

盗む点: テスト式を一時束縛し、真のときだけユーザの名前に渡す。第 13 章のケース A で再実装する。

7.5 通読版での止め方と追記

通読の核は `when / or / when-let` の 3 つ。

追記として `cond` と `→` の展開も `ch07-core-read` で観察できる (形状変換・対句の型)。

ソースを追うときは Clojure 1.12.x の `src/clj/clojure/core.clj` で `defmacro when / or / cond / →` を検索する。

行番号は版で動くため、シンボル名で当たること。

7.6 演習

1. **読む:** 上記 3 マクロ (+任意で `cond / →`) について「盗む点」を各 1 行で書け。

2. **判断:** どれが関数にできないか、一言で述べよ。

たたき台: `src/learn_macro/exercises/ch07.clj` / 解答: `src/learn_macro/answers/ch07.clj` ##
7.7 まとめ—関数でよいのか?

`when` や `when-let` がマクロなのは、評価制御と束縛が本質だからである。
同じ見た目を関数で無理に真似ると、呼び出し側が `thunk` だらけになる。

実行: (`learn-macro.ch07-core-read/demo`)

第 8 章 制御構文を自分で書く—`unless` 再訪

状態: 確定版 v1.0 (工程 11) サンプル: `learn-macro.ch08-unless` (実装は `ch04-defmacro`)

8.1 動機

第 4 章で作った `unless` を、**設計手順つきで見直し**、`doc` と展開の読みやすさを確認する (再掲は意図的である)。

通読版はここで制御構文部分を一旦区切る。独自の `cond` 拡張や `cond→` の自作は、完全版では **第 13 章ケース C** で扱う。

8.2 設計手順 (再掲)

1. 動機を一文で書く (評価制御か?)
2. 関数 + `thunk` で足りるか試す
3. **先に展開形を書く**
4. `defmacro` でテンプレート化する
5. `macroexpand-1` と実行で検証する

8.3 再確認

```
(unless (seq [1]) :hidden) ; => nil
(unless (seq []) :shown)   ; => :shown
```

展開:

```
(if (clojure.core/not (seq [])) (do :shown))
```

docstring が付いていることも確認する (→ #'unless meta :doc)。

8.4 やりすぎの兆候

- 似た分岐マクロがチーム内に増殖している
- 展開結果が読めない
- テストが実行時挙動だけで、展開を見ていない

そのときは [第 14 章](#) のチェックリストに戻る。

8.5 演習

第 4 章の when-not-lite を、本章の設計手順でレビューせよ (展開・doc・命名)。

たたき台: src/learn_macro/exercises/ch08.clj / 解答: src/learn_macro/answers/ch08.clj ##
8.6 まとめ—関数でよいか？

unless は教育用にはよいが、実務では when-not で足りることがほとんどである。
既存の core に無い評価制御が本当に必要かを、追加前に疑え。

実行: (learn-macro.ch08-unless/demo)

読み方の分岐

- **通読版:** このあと [第 9–12 章](#) は飛ばし、[第 13 章](#) (A/B) へ進む。
 - **完全版:** [第 9 章 束縛とリソース](#) へ進み、10→11→12→13→14 と読む。
-

第 9 章 束縛とリソースのマクロ

状態: 確定版 v1.0 (工程 11) サンプル: learn-macro.ch09-binding

9.1 動機

第 8 章までで制御構文の設計手順を見た。次の定番が **束縛マクロ** である。
利用者に「名前とテスト式」を書かせ、内側では let を生成する——その形を先に固定すると、変数捕獲と二重評価を防ぎやすい。

9.2 束縛マクロの定石

先に目標の展開形を書く。

```
;; 目標 (概念)
(let [temp__auto__ <test>]
  (when temp__auto__
    (let [<sym> temp__auto__]
      ...body...)))
```

ポイント:

1. テスト式は **一度だけ** 評価する（一時束縛）
2. ユーザのシンボルは、真のときだけ束縛する
3. 一時名は temp# などで捕獲を避ける

第 7 章の when-let と同じ型である（後述の第 13 章ケース A でも再掲する）。

9.3 if-let-lite / when-let-lite

```
(defmacro when-let-lite
  [[sym test] & body]
  `(let [temp# ~test]
    (when temp#
      (let [~sym temp#]
        ~@body))))

(defmacro if-let-lite
  ([bindings then]
   `(if-let-lite ~bindings ~then nil))
  ([sym test] then else)
  `(let [temp# ~test]
    (if temp#
      (let [~sym temp#] ~then)
      ~else))))
```

clojure.core との差分学習: 本実装は **単一束縛** のみ。複数束縛や破壊的束縛ベクトルの完全再現はしない (教育用の薄い版)。

9.4 リソース管理—with-open の構造

```
(macroexpand-1
  '(with-open [r (clojure.java.io/reader "x.txt")]
    (slurp r)))
```

骨格は「束縛 → body → finally で .close」である。
自作するときは、**必ず閉じる経路**（例外時含む）を展開形で先に書く。

完全な再実装は必須としない。サンプルではミニ版 with-closeable を載せる。

9.5 with-redefs の位置づけ

動的 var を一時的に差し替えるマクロである。
テストで便利だが、並行実行や「本番コードへの持ち込み」には向かない。
束縛の見え方がコード上に現れにくいため、展開と副作用の範囲を特に意識する。

9.6 遅延の糖衣（橋渡し）

delay は **マクロ** である（本体を関数で包み、Delay オブジェクトを作る）。lazy-seq も同様にマクロだ。
それでも「まだ評価したくない」が本質なら、まず既存の delay / lazy-seq / 高階関数で足りるかを疑い、**独自の遅延構文を増やす前に立ち止まる**。

チャネル DSL (core.async の go 等) は変換が重い。初版では「構文と実行モデルが分かれるコスト」を知るに留める。

9.7 定義マクロへの橋渡し

defonce などは「定義の形」を変えるマクロである。
深い実装は選択ケース（定義マクロ）へ送り、ここでは **展開を一度見る** だけでよい。

```
(macroexpand-1 '(defonce answer 42))
```

9.8 演習

1. **読む**: with-open の展開骨格を 3 行で書け。
2. **書く**: 単一束縛の when-let-lite を自分の ns で再実装し、副作用が 1 回だけであることを示せ。
3. **判断**: ファイルを開く処理を、関数 API（呼び出し側が with-open）にするか、独自 with-my-resource を増やすか。

骨組み: src/learn_macro/exercises/ch09.clj

9.9 まとめ—関数でよいか？

束縛とリソースは、**呼び出し構文と評価回数／解放保証**が本質のときにマクロが効く。
単なる「値を受け取って使う」だけなら、関数で足りることが多い。

実行: (learn-macro.ch09-binding/demo)

第 10 章 コード生成の定石とヘルパー関数への委譲

状態: 確定版 v1.0（工程 11） サンプル: learn-macro.ch10-codegen

10.1 動機

マクロ本体にロジックを詰め込むと、テストもデバッグも難しくなる。
定石は **マクロを薄くし、変換は関数に委譲する** ことである。

10.2 テンプレート展開（素直な形）

構文引用で「穴」だけを埋める形が基本である。

```
(defmacro unless
  [test & body]
  `(if (not ~test) (do ~@body)))
```

分岐や走査が増えてきたら、次節へ進む。

10.3 薄いマクロ+関数

```
(defn emit-greeting
  " データ → フォーム。純粋関数としてテストできる。"
  [name-sym]
  `(println (str "hello, " ~(name name-sym))))
```

```
(defmacro greet
  [name-sym]
  (emit-greeting name-sym))
```

- マクロ: 引数（未評価フォーム／シンボル）を受け取り、関数へ渡す
- 関数: データやシンボルを受け取り、生成フォームを返す

展開時に動く関数なので、通常の単体テストで出力フォームを断言しやすい。

10.4 データからコードへ（テーブル駆動）

```
(def op-table
  {'inc 'clojure.core/inc
   'dec 'clojure.core/dec})

(defn emit-op-call
  [op arg-form]
  (let [f (get op-table op)]
    (when-not f
      (throw (ex-info "unknown op" {:op op :known (keys op-table)})))
    `(~f ~arg-form)))

(defmacro op-call
  [op arg]
  (emit-op-call op arg))
```

表を増やすだけで語彙を拡張できる。エラーも「未知の op」として展開時に落とせる（後述の第 12 章へ続く）。

10.5 フォームの行走査—clojure.walk

ネストしたフォームを一括変換したいときに使う。

ただし **意図しない節まで書き換える** 危険がある。

必要な部分だけを関数で組み立てられるなら、clojure.walk は後回しでよい。

10.6 マクロがマクロを生成する

defmacro の本体が、別の defmacro フォームを返す「マクロ工場」は可能だが、複雑度が一気に上がる。初版では **存在と警告** に留め、実務では「薄いマクロ＋関数」を優先する。

10.7 コンパイル時計算

展開時に定数表を作るのは、コストが予測でき再現できる範囲では有用である。

やってはいけない例:

- 展開時にネットワークや時計に依存する
- 環境によって結果が変わる計算を「ゼロコスト」と偽る

10.8 演習

1. **読む:** サンプルの greet / emit-greeting の責務分割を一文で説明せよ。
2. **書く:** 変換関数をテストし、マクロは一行委譲にせよ (src/learn_macro/exercises/ch10.clj)。
3. **判断:** いまの変換に clojure.walk は必要か？ コンパイル時計算は妥当か？

10.9 まとめ—関数でよいか？

生成ロジックの大半は関数でよい。
マクロは「評価前の形を受け取る入口」に薄く置く。

実行: (learn-macro.ch10-codegen/demo)

第 11 章 DSL の設計判断

状態: 確定版 v1.0 (工程 11) サンプル: learn-macro.ch11-dsl

11.1 動機

「DSL だからマクロ」と決め打ちするのは早い。
本章の目標は、**マクロ DSL とデータ駆動 API を比較し、境界の置き方を説明できる**ことである。

11.2 内部 DSL の種類

表 11.1 内部 DSL の型

型	見た目	典型
データ寄り	Clojure の値そのもの	Hiccup ベクトル、HoneySQL マップ
糖衣寄り	少し特別な呼び出し	薄い defroute 風
変換が重い	専用の実行モデル	core.async/go など (観察のみ)

「領域の語彙」を構文にするほど、展開・ツール・教育コストが上がる。

11.3 判断基準

1. **評価制御**が必要か？ (短絡・遅延・束縛)
2. **新しい束縛構文**が必要か？
3. データ + 関数では足りないか？
4. 呼び出し側にマクロ感染を強くないか？

「見た目を領域っぽくしたい」だけでは、マクロの理由として弱い。

11.4 良い境界—入口だけマクロ、中はデータ + 関数

```
;; 中身はデータ
(def routes-data
  [["GET" "/" :home]
   ["GET" "/items/:id" :item]])

(defn compile-routes
  [routes]
  ;; データ → ルータ (関数)。テスト容易
  (into {}
    (map (fn [[method path handler]]
```

```
[[method path] handler]))
routes))
```

```
;; 入口だけ「書き味」を足す（任意）
;; route-forms は展開時のフォーム（リテラル想定）。実行時に計算した表は compile-routes へ直接渡せ。
(defmacro defroutes-lite
  [name & route-forms]
  `(def ~name (compile-routes ~(vec route-forms))))
```

マクロが壊れても、compile-routes とデータは残る。
defroutes-lite に渡す各ルートは **展開時にデータとして埋め込まれる**（典型はリテラルのベクタ）。実行時に組み立てた表を使いたいときは、入口マクロを使わず compile-routes を関数と呼ぶ。

11.5 事例観察（対比）

詳細は ../notes/ライブラリ観察 _L3_L4_L5.md。

表 11.2 定番題材とマクロ度

題材	マクロ度	学ぶ点
Hiccup	低	HTML はデータで足りることが多い
HoneySQL	低	クエリ合成・テストにデータが勝つ
Compojure 型	中～高	構文の魅力と展開コスト
Reitit 型	低寄り	同じ問題域でもマクロ度を下げられる

11.6 悪い DSL の兆候

- 展開結果が読めない
- IDE / リンタが追いつけない
- オンボーディングで「魔法の書き方」を暗記させる
- データとして合成・生成できない

11.7 小さな DSL スケッチ

サンプルでは「ルート表データ＋任意の入口マクロ」を動かす。
HTML をマクロ構文にしたくなったら、まず Hiccup 型（ベクトル）で足りるか問う。

11.8 演習

1. **判断:** 次をマクロ／データ／関数に分類し、一文で理由を書け。
 - (a) 社内 defroute
 - (b) 設定マップを組み立てるだけのもの
 - (c) 短絡する認可チェック構文
2. **書く:** 入口マクロ 1 つ＋ compile-* 関数を用意し、関数側だけをテストせよ。

11.9 まとめ—関数でよいか？

DSL の中心はデータと関数に置き、マクロは **入口の薄皮** に留めるのが壊れにくい。評価制御が本質でないなら、マクロ DSL を選ばない。

実行: (learn-macro.ch11-dsl/demo)

第 12 章 テスト、エラー、ツールチェーン

状態: 確定版 v1.0 (工程 11) サンプル: learn-macro.ch12-tooling

12.1 動機

マクロは「動いた」だけでは足りない。

展開形と**実行時挙動**を分けてテストし、失敗時に利用者が原因を追えるようにする。

12.2 何をテストするか

表 12.1 マクロのテスト層

層	見るもの	例
展開	macroexpand-1 の形	一時束縛があるか、二重評価がないか
実行	戻り値・副作用	短絡したか、1 回だけ評価されたか
エラー	展開時例外のメッセージ	未知のオプション、引数個数

gensym 名は実行ごとに変わるので、完全一致が難しいときは **骨格の正規化** (自動 gensym をプレースホルダに置換) や、重要な部分木だけを断言する。

12.3 展開の単位テスト

```
(defmacro unless [test & body]
  `(if (not ~test) (do ~@body)))

;; 自作マクロは quote だけでと解決されず展開されない (第 5 章の罠)
(= (macroexpand-1 (list `unless false :ok))
   '(if (closure.core/not false) (do :ok)))
```

呼び出しフォームは list と構文引用したマクロ名で組み立ててから展開する。

12.4 エラーメッセージ

展開時に落とす情報は、**利用者が直す手がかり**になる。

```
(defn emit-op-call [op arg-form]
  (let [f (get op-table op)]
    (when-not f
      (throw (ex-info "unknown op"
                       {:op op :known (keys op-table)})))
    `(~f ~arg-form)))
```

(第 10 章の実装と同じ型。未知の語彙を展開時に落とす。)

悪い例: NullPointerException だけ、または内部実装の詳細だけ。

12.5 リンタ・IDE（概要）

clj-kondo はマクロの引数解析に **hooks** を使える。
深い設定は本書の範囲外だが、「チームで使うマクロは解析器に教える必要がある」と知っておく。
教えないと、未使用警告や未解決シンボルがノイズになる。

12.6 バージョンと再現

本書のサンプルは Clojure **1.12.x** (deps.edn は 1.12.1)、推奨 JDK **21**。
展開結果の細部 (gensym 名など) は環境で変わりうるので、**形**をテストする。

12.7（補）ClojureScript

CLJS ではマクロを **.clj / .cljs のマクロ用名前空間**に置き、CLJS 側から参照する配置が一般的である。
詳細は公式ガイドへ任せ、ここでは「配置が JVM と違う」ことだけ押さえる。

12.8 演習

1. **書く**: 自作マクロに展開テストを 2 本（成功形／失敗形）。
2. **直す**: 意味不明なエラーを、ex-info とヒント付きに直せ。

骨組み: src/learn_macro/exercises/ch12.clj

12.9 まとめ—関数でよいか？

テストとエラー設計のコストが高いなら、そもそもマクロが厚すぎるサインである。
薄い入口＋関数なら、通常の間数テストで大半を賄える。

実行: (learn-macro.ch12-tooling/demo)

第 13 章 ケーススタディ（A / B / C）

状態: 確定版 v1.0（工程 11） **サンプル**: learn-macro.ch13-cases
テスト: clojure -M:test

通読版の必須はケース A / B。 **ケース C（簡易 cond→）は完全版**（推奨）。選択ケース（定義マクロ／小さな DSL など）は後続でも可。

13.1 進め方

1. 仕様を一文で書く
 2. 展開形を先に書く
 3. 実装する
 4. 罣（二重評価・捕獲）を潰す
 5. 実行と展開テストを残す
-

13.2 ケース A —unless / when-let-lite

仕様

- unless …第 4・8 章の再掲（評価制御）
- when-let-lite …単一束縛。テスト式は **一度だけ** 評価し、真なら本体へ渡す

実装（要旨）

```
(defmacro when-let-lite
  [[sym test] & body]
  `(let [temp# ~test]
    (when temp#
      (let [~sym temp#]
        ~@body))))
```

展開前後

```
;; before
(when-let-lite [x (first [7])] (inc x))
```

```
;; after
(let [temp__auto__ (first [7])]
  (when temp__auto__
    (let [x temp__auto__]
      (inc x))))
```

検証ポイント

```
(when-let-lite [x (first [7])] (inc x)) ; => 8
(when-let-lite [x nil] (inc x))         ; => nil
```

```
;; テスト式の副作用は 1 回
(let [n (atom 0)]
  (when-let-lite [x (do (swap! n inc) @n)] x)
  @n) ; => 1
```

clojure.core/when-let と同じ型を、小さな実装で体感するのが目的である。

13.3 ケース B —labeled-time

仕様

式を一度評価し、経過時間 (ms) ・ 戻り値 ・ ラベルを返す。

```
(defmacro labeled-time [label expr]
  `(let [result# (good-time ~expr)]
    (assoc result# :label ~label)))
```

good-time は第 6 章の衛生的な計測マクロである。

一段の macroexpand-1 では外側が let になった時点で止まり、**内側の good-time は残る**（先頭だけを見るため）。

内側まで見るときは macroexpand-all か、(good-time …) 部分だけを取り出して macroexpand-1 する。

実行例

```
(labeled-time "sum" (reduce + (range 1000)))  
;; => {:ret 499500, :ms ..., :label "sum"}
```

13.4 ケース C —cond→lite (完全版)

仕様

初期値 `expr` を **一度だけ** 評価し、続く `test / form` の対 (対句) について、`test` が真ならその時点の値を `form` へ → でスレッドする。

`test` にはスレッド中の値は渡さない (`clojure.core/cond→` と同じ契約)。

```
(cond->lite 1  
  true  inc  
  false (* 10)  
  true  (* 3))  
;; => 6
```

展開形 (先に書く)

```
;; 概念 (core に近い let 再束縛)  
(let [cond__auto__ 1  
      cond__auto__ (if true (-> cond__auto__ inc) cond__auto__)  
      cond__auto__ (if false (-> cond__auto__ (* 10)) cond__auto__)  
      cond__auto__ (if true (-> cond__auto__ (* 3)) cond__auto__)]  
  cond__auto__)
```

実装 (要旨)

第 10 章の型どおり、**生成は関数・入口だけマクロ**にする。

```
(defn emit-cond->  
  [expr clauses]  
  (when-not (even? (count clauses))  
    (throw (ex-info "cond->lite expects even number of test/form clauses"  
      {:count (count clauses)})))  
  (let [g (gensym "cond")  
        steps (map (fn [[test step]]  
          `(if ~test (-> ~g ~step) ~g))  
          (partition 2 clauses))]  
    `(let [~g ~expr  
          ~@(mapcat (fn [step] [g step]) steps)]  
      ~g)))  
  
(defmacro cond->lite  
  [expr & clauses]  
  (emit-cond-> expr clauses))
```

検証ポイント

```
(= 6 (cond->lite 1 true inc false (* 10) true (* 3)))  
(= (cond-> 1 true inc false (* 10) true (* 3))  
  (cond->lite 1 true inc false (* 10) true (* 3)))
```

```
;; expr の副作用は 1 回
(let [n (atom 0)]
  (cond->lite (do (swap! n inc) @n) true inc)
  @n) ; => 1
```

条項 (test / form) の個数が奇数で対句にならないときは、展開時に ex-info で落とす (第 12 章のエラー設計と接続)。

学びどころ

表 13.1 ケース C の学びどころ

点	内容
コード形状	対句を if +→ の列へ変換する
衛生	累積値は gensym (cond_...)
二重評価	初期 expr を一度束縛してから使う
薄いマクロ	emit-cond→ を単体で試せる

実務では clojure.core/cond→ を使い、自作は「同じ型を再現できたか」の確認に留める。

13.5 振り返り

表 13.2 ケース A/B/C の境界

ケース	マクロが必要だった点	関数で足りた点
A	評価制御 + 束縛の構文	一時値の計算そのもの
B	式を「書く場所」で一度だけ評価	時間計測の算術
C	対句構文 → if/→ への形状変換	各ステップの計算そのもの

13.6 演習

1. when-let-lite を二重評価する間違った版をわざと書き、テストで撃て。
2. labeled-time のラベルについて判断せよ。現状の ~label は **実行時に評価される式**です。では、(a) 展開時に決まる定数だけ許す設計、(b) 計測結果からラベルを後付けする設計、では引数の置き方がどう違うか？
3. **(ケース C)** cond→lite の対句を 1 組減らした／増やしたときの展開を手で書き、emit-cond→ の結果と照合せよ。条項が奇数個で例外になることも確認せよ。

たたき台: src/learn_macro/exercises/ch13.clj / 解答: src/learn_macro/answers/ch13.clj

選択ケース・anaphoric のメモ: ../notes/選択ケースと anaphoric メモ.md ## 13.7 まとめ—関数でよいのか？

A/B/C いずれも「呼び出し構文と評価回数／コード形状」が本質だった。
ログの数値計算や、すでに手元にある値への if +→ の適用だけなら、関数で十分なことが多い。
cond→ 相当が必要なら、まず **core を使う**。自作は学習と、本当に違う契約が要するときだけ。

選択ケース (定義マクロ／小さな DSL／入口のみ) はレポート形式でも可 (../構成設計.md の第 13 章・選択ケース)。

実行: (learn-macro.ch13-cases/demo) / **テスト:** clojure -M:test

第 14 章 アンチパターンとチェックリスト（通読版短縮）

状態: 確定版 v1.0（工程 11） サンプル: learn-macro.ch14-checklist

14.1 動機

マクロを増やすほど、読む人・ツール・オンボーディングのコストが上がる。
通読版・完全版とも、締めは **書く前と書いた後のチェックリスト** である。

14.2 書く前

1. 評価順・短絡が本質か？
2. 新しい構文／束縛が必要か？
3. データ＋関数では足りないか？
4. 呼び出し側にマクロ感染を強くないか？

一つでも「いいえ」なら、まず関数またはデータ API を設計する。

14.3 書いた後

1. 変数捕獲はないか？ (gensym / x#)
2. 二重評価はないか？ (let で一度)
3. 展開結果は読めるか？
4. 展開テストはあるか？
5. 失敗時のメッセージは分かるか？

14.4 よくある失敗（短く）

表 14.1 よくある失敗と症状

失敗	症状
マクロ感染 第一級扱い データで足りるもののマクロ化	使う側もマクロや特殊な呼び出しを強いられる マクロを値として map しようとして詰まる HTML / SQL / ルートを全部構文にしてしまう

対比の観察は ../notes/ライブラリ観察 _L3_L4_L5.md (Hiccup / HoneySQL / ルーティング)。

14.5 演習（判断）

実務でありがちな次の案を、チェックリストで判定せよ（各 2-3 行）。

1. 社内の defroute マクロ
2. 設定マップを生成するだけの defconfig マクロ

3. テスト用に式の実行時間を測るマクロ
4. レコードのアクセサを一括生成するマクロ
5. if の糖衣をチーム独自に増やすマクロ

たたき台: `src/learn_macro/exercises/ch14.clj` / 解答: `src/learn_macro/answers/ch14.clj` ##

14.6 まとめ—本書の約束の再確認

- **読む** …展開を見る
- **書く** …衛生を守る
- **控える** …関数とデータの境界を先に決める

通読版はここで一区切り。完全版では束縛・DSL・ツール章が続く。

実行: (`learn-macro.ch14-checklist/demo`)

付録 A —ClojureScript 補注

版: N-10 (確定版への追記)

位置づけ: 本書本文は **JVM Clojure** を前提とする。CLJS でマクロを触るときの差分だけを短く置く。完全な CLJS ガイドではない。

A.1 まず押さえること

1. **マクロはコンパイル時だけ**
ブラウザ上の実行時に `defmacro` が増えるわけではない。展開は `cljs` コンパイラ（または `self-host` のコンパイル段階）で起きる。
2. **マクロ定義の置き場**
よくある形は `.clj` / `.cljs` にマクロを置き、`.cljs` から (`:require-macros …`) または `.cljs` のリーダー条件で参照する。
3. **.cljs とリーダー条件**
`#?(:clj … :cljs …)` でホスト差を分ける。マクロ本体が両ホストで意味を持つか、`clj` 側だけの展開か、を意識する。

A.2 JVM Clojure との差（実務で刺さる点）

点	JVM Clojure (本書)	ClojureScript
評価の場	同じプロセスで REPL 展開しやすい	コンパイル単位・モジュール境界が強い
名前空間	<code>ns</code> とマクロが同居しやすい	マクロ専用 <code>ns/require-macros</code> が必要になりやすい
相互運用	Java クラス <code>macroexpand</code> REPL で即試せる	JS オブジェクト・ <code>goog/npm</code> ツールやコンパイルログ、 <code>cljs</code> REPL の手順が環境依存
自己ホスト	通常は不要	<code>self-hosted cljs</code> では「誰が展開するか」がさらに分岐

本書の衛生・二重評価・展開を見る型は **そのまま使える**。変わるのは主に **モジュールとコンパイルの手続き** である。

A.3 やってよい／控える（CLJS 文脈）

- **よい:** 薄い糖衣、評価制御、.cljsc で共有する小さなマクロ
 - **控える:** JS ランタイムの都合を隠す巨大 DSL、マクロ感染で cljs 側の解析・最適化を壊すこと
 - **データ優先:** Hiccup 的 UI やクエリがデータで足りるなら、CLJS でもマクロにしない（第 11・14 章と同じ判断）
-

A.4 最小スケッチ（イメージ）

```
;; macros.cljs (概念例)
(ns learn-macro.cljs-sketch.macros)
```

```
(defmacro unless [test & body]
  `(if (not ~test) (do ~@body)))
```

```
;; usage.cljs (概念例)
(ns learn-macro.cljs-sketch.usage
  (:require-macros [learn-macro.cljs-sketch.macros :refer [unless]]))
```

実際のプロジェクトではビルドツール（shadow-cljs / figwheel 等）のマクロ解決ルールに従う。

A.5 この付録の止め方

- analyzer やコンパイラ内部、高度な self-host は扱わない（企画の「書かないこと」）
 - 詳細は公式の ClojureScript ドキュメントと、利用中のビルドツールのマクロ節へ
- 本文の演習・ケーススタディは JVM 上の deps.edn プロジェクトで続けるのが確実である。
-